E-Commerce Management System

Project Report

Technology: MERN Stack

MongoDB • Express.js • React.js • Node.js

Project	E-Commerce Management System
Developer	Ali Asghar
Frontend	React.js, Redux Toolkit, React Router, Tailwind CSS
Backend	Node.js, Express.js
Database	MongoDB with Mongoose
Authentication	JWT with HTTP-only cookies
Payment	Payment gateway not integrated

Repository: github.com/aliasghar-codes/Ecommerce-Management-System-Using-MERN-Stack

Prepared as a concise technical overview of the implemented project.

1. Project Overview

The E-Commerce Management System is a full-stack web application developed using the MERN stack. It provides a complete online shopping workflow where customers can create an account, browse products, search by product or category, view product details, manage a shopping cart, place orders and submit reviews. A separate admin panel is included to manage products, categories, coupons, orders, messages and basic store information.

The application is divided into a React frontend and a Node.js/Express backend. MongoDB is used to store users, products, categories, carts, orders, coupons, reviews and contact messages. The backend exposes REST API routes under the **/api/v1** prefix and applies authentication middleware where required.

Project Purpose

The main purpose of the project is to provide a practical e-commerce system rather than a simple product listing page. It demonstrates how a modern frontend communicates with a backend API, how data is stored in MongoDB, how users are authenticated, and how an administrator can manage the main parts of an online store.

2. Objectives

- Build a working e-commerce website using the MERN stack.
- Provide a simple and responsive shopping experience for customers.
- Implement user registration, login and logout with protected operations.
- Allow customers to browse, search and explore products by category.
- Provide shopping cart and order placement functionality.
- Add product reviews and ratings for customer feedback.
- Create an admin panel for store and order management.
- Use MongoDB for persistent storage and Express APIs for communication.
- Keep the system modular so more features can be added later.

3. Main Features

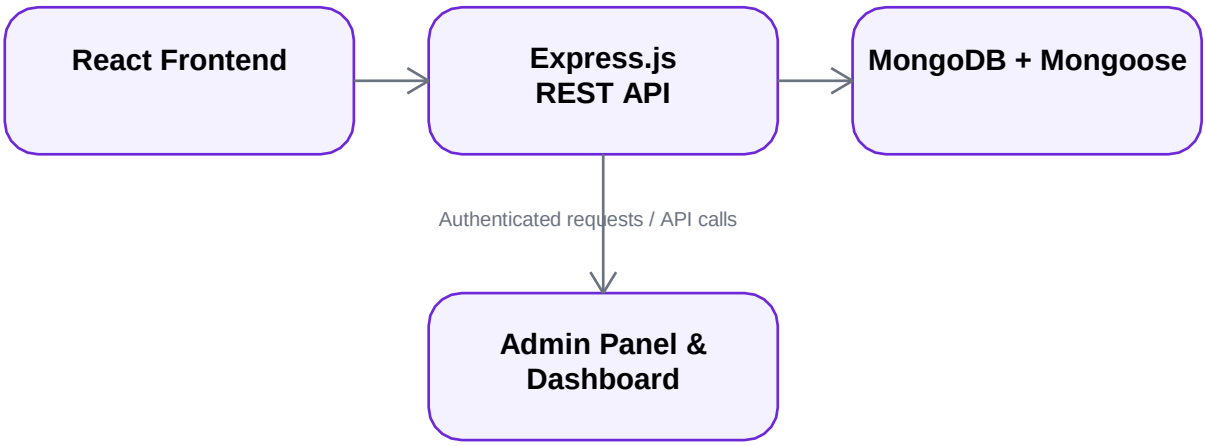
Area	Implemented functionality
Customer account	Registration, login and logout with JWT-based authentication.
Product browsing	Home sections, product listing, product details, related products and category-based browsing.
Search	Search products by name and search products through category names.
Shopping cart	Add products, view cart, update cart, remove an item and clear the cart.
Orders	Checkout/order form, order creation and order status handling.
Reviews	Authenticated users can add one review per product with a 1–5 rating.
Coupons	Admin can create, view and delete discount coupons.
Admin panel	Dashboard, product management, categories, coupons, orders, messages and admin registration.
Contact	Logged-in users can send messages and admins can view/delete messages.

4. Technology Stack

Technology	Use in the project
MongoDB	Stores application data such as users, products, orders, carts and reviews.
Mongoose	Defines MongoDB schemas and handles database operations.
Express.js	Creates the REST API and connects frontend requests to backend logic.
Node.js	Runs the backend server.
React.js	Builds the customer-facing and admin user interfaces.
Redux Toolkit	Manages frontend application state, including authentication-related state.
React Router	Handles navigation and protected application routes.
Axios	Used for HTTP communication between frontend and backend.
Tailwind CSS	Provides utility-based styling for the frontend.
JWT	Provides access tokens for authenticated users/admins.
bcrypt	Hashes user passwords before storing them.
React Toastify	Displays user-friendly success and error messages.
Vite	Development and build tool for the React frontend.

5. System Architecture

The system follows a client-server architecture. The React frontend sends HTTP requests to the Express API. The API performs validation, authentication and business operations before reading or writing data through Mongoose. MongoDB stores the persistent data.



The backend uses separate route files for users, admins, products, categories, contacts, dashboard information, coupons, reviews, cart operations and orders. This separation keeps the API easier to understand and maintain.

6. Customer Module

- **Registration and Login:** Users create an account with a username, email and password. Passwords are hashed before storage, and successful authentication produces a JWT access token.
- **Product Discovery:** Customers can view products, open individual product pages, search by name and browse products by category.
- **Product Details:** Product information includes name, description, price, quantity, category, size, gender, colors and product images.
- **Cart:** Authenticated users can add products, view cart contents, update quantities/data, remove individual items and clear the cart.
- **Reviews:** A logged-in user can submit a review containing a rating from 1 to 5 and review text. The backend prevents the same user from adding another review for the same product.
- **Orders:** Customers provide delivery and order information and place an order. The current implementation stores the selected payment method and order details but does not process online payments.

7. Admin Module

- **Admin Authentication:** Separate admin login and registration routes are provided.
- **Dashboard:** The dashboard can show total registered users, pending orders, total products, messages, active coupons and total sales from completed orders.
- **Product Management:** Admins can add products with multiple images and can list or delete products.
- **Category Management:** Admins can create, list and delete categories.
- **Coupon Management:** Admins can create coupons with a code, discount percentage, start date and end date, and can list or delete them.
- **Order Management:** Admins can view all orders and update an order's status such as pending, shipped, completed or canceled.
- **Message Management:** Admins can view customer contact messages and delete individual or multiple messages.

8. API Structure

Base route	Purpose
/api/v1/user	Customer registration, login and logout
/api/v1/admin	Admin login and registration
/api/v1/product	Product creation, deletion, listing, search and details
/api/v1/category	Category creation, listing and deletion
/api/v1/dashboard	Admin dashboard statistics
/api/v1/coupon	Coupon creation, listing and deletion
/api/v1/review	Product reviews
/api/v1/cart	Cart operations
/api/v1/bill	Order creation, listing and status updates
/api/v1/contact	Customer messages and admin message management

9. Database Design

MongoDB is used as the main database. Mongoose schemas define the structure of the application's data. The main collections/models identified in the project are shown below.

Model	Important data
User	Username, email, hashed password, role and timestamps
Product	Name, description, price, category, quantity, size, gender, colors and images
Category	Product category information
Cart	User-specific cart and selected product information
Order	Customer details, products, total amount, payment method, status and timestamps
Coupon	Code, discount percentage, start date and end date
Review	User, product, name, rating and review text
Contact	Messages submitted by authenticated users

10. Authentication and Security

- Passwords are hashed with bcrypt before being stored in MongoDB.
- JWT tokens are used to identify authenticated users and admins.
- Authentication middleware protects operations such as cart actions, reviews and order creation.
- Separate admin authentication middleware protects admin-only routes.
- Authentication cookies are configured as HTTP-only, secure and SameSite Strict in the backend.
- Environment variables are used for sensitive configuration such as database connection and token settings.

11. Order and Payment Flow

The current project supports the order creation process but does not include a third-party payment gateway. The order model contains a paymentMethod field and uses **Cash on delivery** as its default value. When a customer places an order, the backend stores the customer information, selected products, total amount and payment method. An administrator can then update the order status.

Important limitation: No Stripe, PayPal, JazzCash, Easypaisa or other online payment gateway is integrated in this version. Adding an online payment service would be a future enhancement rather than an existing feature.

12. Project Workflow

Step	Process
1	Customer opens the React web application.
2	Customer registers or logs in.
3	Products are requested from the Express API.
4	Customer searches, browses or opens a product.
5	Customer adds selected products to the cart.
6	Customer reviews the cart and submits order information.
7	Express API creates and stores the order in MongoDB.
8	Admin views the order from the admin panel.
9	Admin changes the order status as it is processed.

13. Testing Approach

The project was checked around the main user and admin flows. The following areas are the key functional tests for the system:

Test area	Expected result
User registration	A new customer account is created and duplicate email registration is rejected.
User login	Valid credentials authenticate the user and invalid credentials are rejected.
Product search	Matching products are returned for a product name or category search.
Cart	Products can be added, updated, removed and cleared.
Review	Authenticated users can add a 1–5 rating and review text; duplicate review is blocked.
Order	A valid order is stored with customer, product and total information.
Admin product control	Admin can add, list and delete products.
Admin orders	Admin can view orders and update their status.
Coupons	Admin can create, view and delete coupons.
Messages	Authenticated customers can send messages and admins can manage them.

14. Current Limitations

- There is no online payment gateway in the current version.
- The project is focused on core e-commerce functionality and does not include advanced features such as delivery-service integration, automated email notifications or real-time order tracking.
- The dashboard provides useful store counts and sales totals, but it is not a full business analytics system.

15. Future Improvements

- Integrate a secure online payment gateway suitable for the target market.
- Add wishlist functionality and saved products.
- Add customer order-history pages and clearer order tracking.
- Add email/SMS notifications for order updates.
- Improve dashboard analytics with sales charts, filters and date ranges.
- Add stronger validation, automated tests and rate limiting.
- Deploy the frontend, backend and database with production-ready environment configuration.

16. Conclusion

The E-Commerce Management System demonstrates the practical use of the MERN stack to build a complete web-based shopping platform. It covers the main flow from customer authentication and product discovery to cart management and order placement, while also providing an admin panel for managing important store operations. The project uses a clear separation between the React frontend, Express API and MongoDB database.

Although an online payment gateway is not included, the project provides a solid base for extending the system with payment processing, better analytics, notifications and additional customer features. Overall, it demonstrates practical experience with React, REST APIs, MongoDB, authentication, state management and admin-side application development.

17. Project Information

Developer	Ali Asghar
Project	E-Commerce Management System Using MERN Stack
Repository	https://github.com/aliasghar-codes/Ecommerce-Management-System-Using-MERN-Stack
License	MIT License
Primary stack	MongoDB, Express.js, React.js, Node.js
Payment gateway	Not integrated in the current version